

Laboratorium -- dokumentowe bazy danych: Couchbase

Temat: Couchbase, dokumenty JSON, indeksy, JOIN, UNNEST i analiza danych Northwind

Baza: Couchbase Community uruchomiony w Dockerze

Bucket: north0

Scope: _default

Główne kolekcje: orders, orderdetails, customers, products, orders_nested

Imię i nazwisko:

- Paweł Moćko
- Katarzyna Wesołowska **Grupa:** Grupa 5.

Cel ćwiczenia

Po wykonaniu laboratorium student powinien umieć:

1. uruchomić i zweryfikować środowisko Couchbase,
2. poruszać się po panelu Couchbase i korzystać z Query Workbench,
3. rozumieć strukturę bucket → scope → collection,
4. wykonać podstawowe zapytania SQL++ / N1QL na dokumentach JSON,
5. wyjaśnić, dlaczego Couchbase wymaga indeksów do wykonywania zapytań,
6. odróżnić primary index od indeksu celowego,
7. wykonać JOIN między kolekcjami dokumentów,
8. porównać model niezagnieżdżony z modelem zagnieżdżonym,
9. użyć UNNEST do rozbijania tablicy zagnieżdżonej w dokumencie,
10. wykorzystać EXPLAIN do podstawowej interpretacji planu zapytania.

Ważne informacje

W tym laboratorium pracujemy na danych Northwind załadowanych do Couchbase.

Dane są dostępne w dwóch wariantach modelowania:

Model niezagnieżdżony

Dane są podzielone na osobne kolekcje:

- orders -- zamówienia,
- orderdetails -- pozycje zamówień,
- customers -- klienci,

- products -- produkty.

W tym wariancie, aby połączyć zamówienia z pozycjami zamówień, używamy JOIN.

Model zagnieżdżony

Dodatkowo przygotowana jest kolekcja:

- orders_nested.

W tej kolekcji jeden dokument odpowiada jednemu zamówieniu, a pozycje zamówienia są zapisane wewnątrz dokumentu w tablicy items. W tym wariancie do rozbicia pozycji zamówienia używamy UNNEST.

Jak korzystać ze ściąg

Do laboratorium dołączona jest ściąg Couchbase_SQLPP_sciaga.md. Korzystaj z niej jak z dokumentacji pomocniczej: sprawdzaj składnię JOIN, UNNEST, IFMISSINGORNULL, CREATE INDEX i EXPLAIN, ale nie kopiuj bezrefleksyjnie gotowych rozwiązań. Oceniane są również komentarze i interpretacja.

Sprawozdanie

Oddawane sprawozdanie powinno zawierać:

- kod zapytań,
- wyniki zapytań jako tabele albo zrzuty ekranu,
- krótkie komentarze interpretacyjne,
- odpowiedzi na pytania wskazane w zadaniach.

Format sprawozdania: PDF albo Markdown.

Kod SQL++ / N1QL formatuj jako bloki kodu.

Nie oddawaj samych zrzutów ekranu bez komentarza.

Punktacja

Zadanie Temat Punkty

0 Gotowość środowiska 0

1 Pierwsze poznanie 1 Couchbase i danych

2 Indeksy: brak indeksu, 2 primary index, secondary index

3 JOIN na kolekcjach 2 dokumentów

4 Model niezagnieżdżony 2 vs zagnieżdżony: JOIN vs UNNEST

5 Agregacja biznesowa 1

6 EXPLAIN i refleksja 2 końcowa

Razem 10

0. Gotowość środowiska -- 0 pkt

To zadanie nie jest punktowane, ale jest warunkiem rozpoczęcia pracy.

Wykonaj

1. Uruchom środowisko:

```
docker compose --profile init up -d
```

2. Sprawdź, czy działają kontenery:

```
docker ps
```

3. Wejdź do panelu Couchbase:

```
http://localhost:8091
```

4. Zaloguj się:

Login: student Hasło: student

5. Sprawdź, czy widzisz bucket:

```
north0
```

6. Wejdź do Query Workbench i uruchom:

```
SELECT 1 AS test;
```

Do sprawozdania

Nie trzeba dołączać pełnych logów. Wystarczy jedno zdanie:

Środowisko Couchbase zostało uruchomione, logowanie działa, bucket north0 jest widoczny, a Query Workbench wykonuje zapytania.

1. Pierwsze poznanie Couchbase i danych -- 1 pkt

Cel

Poznaj podstawową strukturę danych w Couchbase i sprawdź, jakie kolekcje są dostępne.

Wykonaj

1. W panelu Couchbase za pomocą zakładek w menu bocznym np. 'Buckets', 'Documents', 'Query' zbadaj strukturę:

bucket → scope → collection

W bazie danych zdefiniowany jest jeden bucket: `north0`, w nim jeden scope: `_default`, w którym zdefiniowane są kolekcje:

- `_default`
- `categories`
- `customers`
- `employees`
- `ordedetails`
- `orders`
- `orders_nested`
- `products`
- `shippers`
- `suppliers`

2. W Query Workbench policz liczbę dokumentów w kolekcjach:

- `orders`,
- `orderdetails`,
- `customers`,
- `products`,
- `orders_nested`.

W poprawnie zainicjalizowanym środowisku powinieneś otrzymać orientacyjnie:

kolekcja	liczba dokumentów
order	830
orderdetails	2150
customers	91
products	77
order_nested	830

Powyżej: uzupełnione wyniki na podstawie wyników zapytań formy `SELECT COUNT(*) FROM nazwa_kolekcji`

3. Podejrzyj kilka dokumentów z kolekcji orders.

```
[
  {
    "o": {
      "CustomerID": "VINET",
      "EmployeeID": 5,
      "Freight": 32.38,
      "OrderDate": {
        "$date": "1996-07-04T00:00:00Z"
      },
      "OrderID": 10248,
      "RequiredDate": {
        "$date": "1996-08-01T00:00:00Z"
      },
      "ShipAddress": "59 rue de l'Abbaye",
      "ShipCity": "Reims",
      "ShipCountry": "France",
      "ShipName": "Vins et alcools Chevalier",
      "ShipPostalCode": "51100",
      "ShipRegion": null,
      "ShipVia": 3,
      "ShippedDate": {
        "$date": "1996-07-16T00:00:00Z"
      },
      "_id": {
        "$oid": "63a060b9bb3b972d6f4e1fc6"
      }
    }
  },
  {
    "o": {
      "CustomerID": "TOMSP",
      "EmployeeID": 6,
      "Freight": 11.61,
      "OrderDate": {
        "$date": "1996-07-05T00:00:00Z"
      },
      "OrderID": 10249,
      "RequiredDate": {
        "$date": "1996-08-16T00:00:00Z"
      },
      "ShipAddress": "Luisenstr. 48",
      "ShipCity": "Münster",
      "ShipCountry": "Germany",
      "ShipName": "Toms Spezialitäten",
      "ShipPostalCode": "44087",
      "ShipRegion": null,
      "ShipVia": 1,
      "ShippedDate": {
```

```
    "$date": "1996-07-10T00:00:00Z"
  },
  "_id": {
    "$oid": "63a060b9bb3b972d6f4e1fc7"
  }
}
]
```

4. Podejrzyj jeden dokument z kolekcji orders_nested.

```
[
  {
    "o": {
      "CustomerID": "VINET",
      "EmployeeID": 5,
      "OrderDate": {
        "$date": "1996-07-04T00:00:00Z"
      },
      "OrderID": 10248,
      "ShipCity": "Reims",
      "ShipCountry": "France",
      "ShipName": "Vins et alcools Chevalier",
      "items": [
        {
          "Discount": 0,
          "LineValue": 98,
          "ProductID": 42,
          "Quantity": 10,
          "UnitPrice": 9.8
        },
        {
          "Discount": 0,
          "LineValue": 168,
          "ProductID": 11,
          "Quantity": 12,
          "UnitPrice": 14
        },
        {
          "Discount": 0,
          "LineValue": 174,
          "ProductID": 72,
          "Quantity": 5,
          "UnitPrice": 34.8
        }
      ],
      "type": "order_nested"
    }
  }
]
```

Wskazówki

Przykład zapytania liczącego dokumenty:

```
SELECT COUNT(1) AS orders_count
FROM `north0`._default.orders
WHERE OrderID IS NOT MISSING;
```

Przykład podejrzenia dokumentu:

```
SELECT o
FROM `north0`._default.orders AS o
WHERE o.OrderID IS NOT MISSING
LIMIT 3;
```

W komentarzu napisz

- Co oznaczają pojęcia bucket, scope i collection?
 - bucket: jest to odpowiednik Bazy Danych z wielu innych systemów bazodanowych jak np. PostgreSQL, ClickHouse.s
 - scope: jest to zgrupowanie kolekcji.
 - collection: jest to zgrupowanie dokumentów, które można indeksować w ramach kolekcji. Nie wymusza się określonego ich kształtu, niemniej kolekcja ma mieć charakter logiczny, tj. w różnych kolekcjach powinny się znaleźć dane dot. produktów oraz klientów.
- Ile dokumentów znajduje się w kolekcjach orders, orderdetails, customers, products i orders_nested?

kolekcja	liczba dokumentów
order	830
orderdetails	2150
customers	91
products	77
order_nested	830

- Czym różni się dokument z kolekcji orders od dokumentu z kolekcji orders_nested? W kolekcji **orders_nested** zawarte są dokumenty, które reprezentują bardziej złożoną strukturę danych, tj. pod kluczem **items** występuje lista obiektów. Kolekcja **orders** obejmuje dokumenty o płaskiej strukturze. Przykładowy dokument z **orders**:

```
{
  "orders": {
    "CustomerID": "VINET",
```

```
"EmployeeID": 2,
"Freight": 1.15,
"OrderDate": {
  "$date": "1996-09-02T00:00:00Z"
},
"OrderID": 10295,
"RequiredDate": {
  "$date": "1996-09-30T00:00:00Z"
},
"ShipAddress": "59 rue de l'Abbaye",
"ShipCity": "Reims",
"ShipCountry": "France",
"ShipName": "Vins et alcools Chevalier",
"ShipPostalCode": "51100",
"ShipRegion": null,
"ShipVia": 2,
"ShippedDate": {
  "$date": "1996-09-10T00:00:00Z"
},
"_id": {
  "$oid": "63a060b9bb3b972d6f4e1ff5"
}
}
```

Przykładowy dokument z `orders_nested`

```
{
  "orders_nested": {
    "CustomerID": "VINET",
    "EmployeeID": 5,
    "OrderDate": {
      "$date": "1996-07-04T00:00:00Z"
    },
    "OrderID": 10248,
    "ShipCity": "Reims",
    "ShipCountry": "France",
    "ShipName": "Vins et alcools Chevalier",
    "items": [
      {
        "Discount": 0,
        "LineValue": 98,
        "ProductID": 42,
        "Quantity": 10,
        "UnitPrice": 9.8
      },
      {
        "Discount": 0,
        "LineValue": 168,
        "ProductID": 11,
        "Quantity": 12,
        "UnitPrice": 14
      }
    ]
  }
}
```



```
    },  
    {  
      "Discount": 0,  
      "LineValue": 174,  
      "ProductID": 72,  
      "Quantity": 5,  
      "UnitPrice": 34.8  
    }  
  ],  
  "type": "order_nested"  
}  
},
```

Dodatkowo dokumenty z kolekcji `orders` mają więcej pól. Czyli dokumenty z kolekcji `orders` zawierają szczegółowe informacje dot. przesyłki, kiedy to `orders_nested` zawiera ich mniej, ale za to zawiera informacje dot. produktów wchodzących w skład zamówienia.

bucket, scope, collection

2. Indeksy: brak indeksu, primary index, secondary index -- 2 pkt

Cel

Zobacz, że Couchbase nie wykonuje zapytań bez odpowiedniego indeksu. Następnie porównaj indeks główny z indeksem celowym.

Część A -- zapytanie bez indeksu

Wykonaj zapytanie:

```
SELECT  
  c.Country,  
  COUNT(1) AS customers_count  
FROM `north0`._default.customers AS c  
GROUP BY c.Country  
ORDER BY customers_count DESC;
```

Oczekiwane zachowanie

W świeżo uruchomionym środowisku zapytanie powinno zakończyć się błędem informującym o braku indeksu. Jeżeli zapytanie działa od razu, oznacza to najczęściej, że w środowisku pozostał wcześniej utworzony indeks.

Zgodnie z przewidywaniami otrzymaliśmy komunikat dot. braku indeksu.

Część B -- primary index

Utwórz primary index:

```
CREATE PRIMARY INDEX idx_customers_primary  
ON `north0`._default.customers;
```

Powtórz zapytanie z części A.

Fragment wyniku:

```
[  
  {  
    "Country": "USA",  
    "customers_count": 13  
  },  
  {  
    "Country": "France",  
    "customers_count": 11  
  },  
  {  
    "Country": "Germany",  
    "customers_count": 11  
  },  
]
```

Część C -- indeks celowy

Usuń primary index:

```
DROP INDEX idx_customers_primary  
ON `north0`._default.customers;
```

Utwórz indeks celowy na polu Country:

```
CREATE INDEX idx_customers_country  
ON `north0`._default.customers(Country);
```

Wykonaj zapytanie z warunkiem indeksowym:

```
SELECT
  c.Country,
  COUNT(1) AS customers_count
FROM `north0`._default.customers AS c
WHERE c.Country IS NOT MISSING
GROUP BY c.Country
ORDER BY customers_count DESC;
```

Oczywiście zyskałobyśmy identyczne rezultaty dla zapytania (pod względem zawartości odpowiedzi serwera bazodanowego)

W komentarzu napisz

- Co się stało przy próbie wykonania zapytania bez indeksu? Otrzymaliśmy błąd, który wskazywał na konieczność zdefiniowania indeksu
- Czym różni się primary index od indeksu celowego? Primary index służy do indeksowania kolekcji według klucza dokumenty, tzn. pola unikalnie identyfikującego go w ramach kolekcji. Główny indeks służy do indeksowania jednego lub wielu pól. Zatem jest przydatny, kiedy oczekujemy filtrowania danych w kolekcji. Na przykład dla zapytania

```
SELECT username from users WHERE email == "abc@def.ghi";
```

indeksowanie według pola email lub klucza złożonego (`email`, `username`) znacznie przyspieszy czas wykonania zapytania.

- Dlaczego w zapytaniu po utworzeniu indeksu celowego dodano warunek WHERE c.Country IS NOT MISSING? CouchBase domyślnie nie indeksuje dokumentów, dla których wartość pola wchodzącego w skład klucza indeksu jest brakująca¹. W przypadku braku tej klauzuli otrzymamy komunikat o brakującym indeksie.
- Dlaczego w środowisku produkcyjnym nie powinno się traktować primary index jako rozwiązania docelowego? Nie pozwala ono na efektywne filtrowanie danych. Wykorzystanie go sprowadza się do robienia pełnych skanów dla zapytań, które można byłoby w innym przypadku zoptymalizować. W realnym użyciu indeksy powinny odwzorowywać ścieżki dostępu, gdy wiele zapytań potrzebuje informacji dot. pewnego pola - powinno znaleźć się w indeksie. **3. JOIN na kolekcjach dokumentów -- 2 pkt**

Cel

Zobacz, że Couchbase pozwala wykonywać JOIN podobny do SQL, ale pracuje na dokumentach JSON i kolekcjach, a nie na klasycznych tabelach relacyjnych.

Część A -- zamówienia z nazwą klienta

Dla zamówień pokaż:

- OrderID,

- OrderDate,
- CustomerID,
- CompanyName.

Przed wykonaniem zapytania utwórz indeks potrzebny do łączenia z kolekcją customers:

```
CREATE INDEX idx_customers_customerid
ON `north0`._default.customers(CustomerID);
```

Jeżeli indeks już istnieje, Couchbase zwróci komunikat o istniejącym indeksie. W takiej sytuacji przejdź do kolejnego kroku.

Następnie przygotuj zapytanie łączące orders z customers.

Zapytanie:

```
SELECT o.OrderID, o.OrderDate, c.CustomerID, c.CustomName
FROM `north0`._default.`customers` AS c
JOIN `north0`._default.`orders` AS o ON c.CustomerID = o.CustomerID
```

Początkowo zwróciło:

```
[ { "code": 4330, "msg": "No index available for ANSI join term orders", "query":
"SELECT o.OrderID, o.OrderDate, c.CustomerID, c.CustomName\nFROM
`north0`._default.`customers` AS c \n JOIN `north0`._default.`orders` AS o ON
c.CustomerID = o.CustomerID" } ]
```

Po dodaniu indeksu dla **Orders**:

```
CREATE INDEX idx_orders_customer_id
ON orders(CustomerID);
```

pomyślnie udało nam się wykonać zapytanie.

Część B -- zamówienia i pozycje zamówień

Dla pozycji zamówień pokaż:

- OrderID,
- CustomerID,

- ProductID,
- UnitPrice,
- Quantity,
- Discount.

Wykorzystaj kolekcje:

- orders,
- orderdetails.

Uwaga: indeksy idx_orders_orderid oraz idx_orderdetails_orderid zostały utworzone automatycznie podczas inicjalizacji środowiska. Nie trzeba ich zakładać ręcznie. Jeżeli mimo to spróbujesz je utworzyć, Couchbase poinformuje, że indeks już istnieje -- to nie jest błąd.

Zapytanie:

```
SELECT o.OrderID, o.CustomerID, item.ProductID, item.Quantity, item.Discount
FROM `north0`.`_default`.`orders` AS o
JOIN `north0`.`_default`.`orders_nested` AS o_n ON o.OrderID = o_n.OrderID
UNNEST o_n.items AS item
```

Kilka wyników:

```
[
  {
    "CustomerID": "VINET",
    "Discount": 0,
    "OrderID": 10248,
    "ProductID": 42,
    "Quantity": 10
  },
  {
    "CustomerID": "VINET",
    "Discount": 0,
    "OrderID": 10248,
    "ProductID": 11,
    "Quantity": 12
  },
]
```

Część C -- wartość zamówienia

Policz wartość zamówienia według wzoru:

wartość pozycji = UnitPrice * Quantity * (1 - Discount)

Brak rabatu traktuj jako 0.

Wskazówka: w dokumentach JSON pole Discount może nie istnieć albo mieć wartość null. W Couchbase do obsługi takich sytuacji służy funkcja IFMISSINGORNULL(od.Discount, 0), która zwraca 0, jeżeli pole nie istnieje lub jest puste.

Dla każdego zamówienia oblicz:

- OrderID,
- CustomerID,
- order_value,
- liczbę pozycji zamówienia.

Pokaż 10 zamówień o najwyższej wartości.

Zapytanie:

```
SELECT
  o.OrderID,
  o.CustomerID,
  SUM(item.UnitPrice * item.Quantity * (1 - IFMISSINGORNULL(item.Discount, 0))) AS
order_value,
  COUNT(1) AS item_count
FROM `north0`.`_default`.`orders` AS o
  JOIN `north0`.`_default`.`orders_nested` AS o_n ON o.OrderID = o_n.OrderID
  UNNEST o_n.items as item
GROUP BY o.OrderID, o.CustomerID
ORDER BY order_value DESC
LIMIT 10
```

CustomerID	OrderID	item_count	order_value
QUICK	10865	2	16387.49998714775
HANAR	10981	1	15810
SAVEA	11030	4	12615.05
RATTC	10889	2	11380
SIMOB	10417	4	11188.4
KOENE	10817	4	10952.844978627563
HUNGO	10897	2	10835.240000000002
RATTC	10479	4	10495.6
QUICK	10540	4	10191.7
QUICK	10691	5	10164.8

rezultaty:

W komentarzu napisz

- Czy JOIN w Couchbase przypomina składnię znaną z SQL? Tak, obie mają formę `tab1 JOIN tab2 ON tab1.x = tab2.x`

- Czy różni się takie łączenie od relacji w klasycznej bazie relacyjnej (np. czy baza wymusza klucze obce i spójność relacji tak jak w typowym modelu relacyjnym)? W przypadku baz danych dokumentowych nie ma wymogu posiadania kluczy obcych w dokumentach kolekcji, ani schemat relacji między dokumentami (czy też samych dokumentów) nie jest wymuszony.
- Dlaczego indeks po stronie dołączanej kolekcji jest ważny? Na podstawie wartości po lewej stronie silnik bazy danych wyszukuje odpowiadające wartości pola wg, którego dochodzi do dołączenia kolekcji z prawej strony. Indeks po prawej stronie pozwala nam na efektywne wyszukiwanie dokumentów do dołączenia.
- Czy największe zamówienia mają zawsze największą liczbę pozycji? Nie, wystarczy spojrzeć na trzy największe zamówienia. Mamy tam liczby zamówień 2,1,4 - gdzie 4 odpowiada najmniejszej wartości spośród tych trzech.

4. Model niezagnieżdżony vs zagnieżdżony: JOIN vs UNNEST -- 2 pkt

Cel

Porównaj dwa sposoby modelowania tych samych danych:

1. model niezagnieżdżony: orders + orderdetails,
2. model zagnieżdżony: orders_nested, gdzie pozycje zamówienia są tablicą items.

Część A -- obejrzyj dokument zagnieżdżony

Podejrzyj dokument zamówienia o numerze 10248 z kolekcji orders_nested.

Zwróć uwagę na pole:

items

Odpowiedź

```
SELECT n
FROM `north0`._default.orders_nested AS n
WHERE n.OrderID = 10248;
```

Wynik:

```
[
  {
    "n": {
      "CustomerID": "VINET",
      "EmployeeID": 5,
      "OrderDate": {
        "$date": "1996-07-04T00:00:00Z"
```

```
    },
    "OrderID": 10248,
    "ShipCity": "Reims",
    "ShipCountry": "France",
    "ShipName": "Vins et alcools Chevalier",
    "items": [
      {
        "Discount": 0,
        "LineValue": 98,
        "ProductID": 42,
        "Quantity": 10,
        "UnitPrice": 9.8
      },
      {
        "Discount": 0,
        "LineValue": 168,
        "ProductID": 11,
        "Quantity": 12,
        "UnitPrice": 14
      },
      {
        "Discount": 0,
        "LineValue": 174,
        "ProductID": 72,
        "Quantity": 5,
        "UnitPrice": 34.8
      }
    ],
    "type": "order_nested"
  }
}
```

Komentarz:

W kolekcji `orders_nested` jedno zamówienie jest zapisane jako jeden dokument JSON. Dla zamówienia `10248` widać podstawowe dane zamówienia, takie jak `CustomerID`, `OrderDate`, `ShipCity` i `ShipCountry`. Najważniejsze jest pole `items`, które zawiera tablicę pozycji zamówienia. W tym przypadku tablica ma 3 elementy, czyli zamówienie składa się z 3 produktów.

Część B -- rozbij tablicę items przez UNNEST

Dla zamówienia 10248 pokaż wszystkie pozycje zamówienia z tablicy items.

Wynik powinien zawierać:

- OrderID,
- CustomerID,
- ProductID,

- UnitPrice,
- Quantity,
- Discount,
- LineValue.

Wskazówka składniowa

UNNEST rozbija tablicę zagnieżdżoną w dokumencie na osobne rekordy. Każdy element tablicy staje się oddzielnym wierszem w wyniku:

```
SELECT
n.OrderID,
item.ProductID
FROM `north0`._default.orders_nested AS n
UNNEST n.items AS item
WHERE n.OrderID = 10248;
```

Rozbuduj to zapytanie o pozostałe kolumny wymienione powyżej.

Odpowiedź

```
SELECT
  n.OrderID,
  n.CustomerID,
  item.ProductID,
  item.UnitPrice,
  item.Quantity,
  IFMISSINGORNULL(item.Discount, 0) AS Discount,
  ROUND(item.UnitPrice * item.Quantity * (1 - IFMISSINGORNULL(item.Discount,
0)), 2) AS LineValue
FROM `north0`._default.orders_nested AS n
UNNEST n.items AS item
WHERE n.OrderID = 10248;
```

Wyniki

OrderID	CustomerID	ProductID	UnitPrice	Quantity	Discount	LineValue
10248	VINET	42	9.8	10	0	98
10248	VINET	11	14	12	0	168
10248	VINET	72	34.8	5	0	174

Komentarz

Po użyciu **UNNEST** tablica **items** została rozbita na osobne wiersze. Każdy element tablicy stał się oddzielnym rekordem w wyniku. Dzięki temu można analizować pozycje zamówienia tak jak zwykłe rekordy, mimo że fizycznie znajdują się wewnątrz jednego dokumentu.

**

Część C -- policz wartość zamówień z modelu zagnieżdżonego

Na kolekcji `orders_nested` policz:

- OrderID,
- CustomerID,
- order_value,
- liczbę pozycji.

Użyj **UNNEST**.

Pokaż 10 zamówień o najwyższej wartości.

Odpowiedź

```
SELECT
  n.OrderID,
  n.CustomerID,
  ROUND(SUM(item.UnitPrice * item.Quantity * (1 - IFMISSINGORNULL(item.Discount,
0))), 2) AS order_value,
  COUNT(1) AS items_count
FROM `north0`._default.orders_nested AS n
UNNEST n.items AS item
WHERE n.OrderID IS NOT MISSING
GROUP BY n.OrderID, n.CustomerID
ORDER BY order_value DESC
LIMIT 10;
```

Wyniki

OrderID	CustomerID	items_count	order_value
10865	QUICK	2	16387.5
10981	HANAR	1	15810
11030	SAVEA	4	12615.05
10889	RATTC	2	11380
10417	SIMOB	4	11188.4
10817	KOENE	4	10952.84
10897	HUNGO	2	10835.24
10479	RATTC	4	10495.6

	10540	QUICK		4		10191.7	
	10691	QUICK		5		10164.8	

Komentarz

W tej części policzono wartość zamówienia na podstawie pozycji zapisanych w tablicy `items`. Największą wartość ma zamówienie `10865` klienta `QUICK`, mimo że ma tylko 2 pozycje. To pokazuje, że liczba pozycji nie zawsze oznacza największą wartość zamówienia.

Część D -- porównaj wynik z modelem niezagnieżdżonym

Porównaj wynik z części C z wynikiem otrzymanym wcześniej przez JOIN na orders i orderdetails (zadanie 3C).

Minimum: porównaj wizualnie top 10 zamówień z obu podejść i napisz, czy wyniki są zgodne.

Opcjonalnie: jeżeli chcesz potwierdzić zgodność formalnie, możesz napisać zapytanie z WITH, które porówna wartości zamówień z obu modeli dla wszystkich 830 zamówień.

W komentarzu napisz

- Na czym polega różnica między JOIN i UNNEST?
- Dlaczego w modelu zagnieżdżonym nie trzeba łączyć orders z orderdetails?
- Czy oba podejścia dają ten sam wynik biznesowy?
- Kiedy zagnieżdżanie pozycji zamówienia w dokumencie może być wygodne?
- Kiedy lepiej zostawić dane w osobnych kolekcjach?

Odpowiedź

```
SELECT
  o.OrderID,
  o.CustomerID,
  ROUND(SUM(od.UnitPrice * od.Quantity * (1 - IFMISSINGORNULL(od.Discount, 0))),
2) AS order_value,
  COUNT(1) AS items_count
FROM `north0`._default.orders AS o
JOIN `north0`._default.orderdetails AS od
  ON o.OrderID = od.OrderID
WHERE o.OrderID IS NOT MISSING
  AND od.OrderID IS NOT MISSING
GROUP BY o.OrderID, o.CustomerID
ORDER BY order_value DESC
LIMIT 10;
```

Wyniki

OrderID	CustomerID	items_count	order_value
-----:	-----:	-----:	-----:
10865	QUICK	2	16387.5
10981	HANAR	1	15810
11030	SAVEA	4	12615.05
10889	RATTC	2	11380
10417	SIMOB	4	11188.4
10817	KOENE	4	10952.84
10897	HUNGO	2	10835.24
10479	RATTC	4	10495.6
10540	QUICK	4	10191.7
10691	QUICK	5	10164.8

Komentarz

Wyniki z modelu zagnieżdżonego i niezagnieżdżonego są zgodne. W obu przypadkach otrzymano te same zamówienia, te same wartości `order_value` i tę samą liczbę pozycji `items_count`.

Różnica polega na sposobie dojścia do danych. W modelu niezagnieżdżonym dane są w osobnych kolekcjach `orders` i `orderdetails`, więc trzeba użyć `JOIN`. W modelu zagnieżdżonym pozycje zamówienia są już zapisane w dokumencie zamówienia w tablicy `items`, więc wystarczy użyć `UNNEST`.

`JOIN` łączy dane z dwóch kolekcji, a `UNNEST` rozбивa tablicę znajdującą się wewnątrz jednego dokumentu. Model zagnieżdżony jest wygodny, gdy często pobieramy całe zamówienie razem z jego pozycjami. Osobne kolekcje są lepsze, gdy dane mają być analizowane niezależnie, na przykład przy raportach sprzedaży według produktów albo przy częstych zmianach pozycji zamówienia.

Oba podejścia dają ten sam wynik biznesowy, ale różnią się strukturą danych i sposobem wykonania zapytania.

5. Agregacja biznesowa -- 1 pkt

Cel

Wykonaj prostą analizę biznesową na danych dokumentowych.

Wybierz [jeden]{underline} wariant

Do zaliczenia zadania wybierz jeden wariant. Jeżeli skończysz wcześniej, wykonaj drugi wariant jako ćwiczenie dodatkowe.

Wariant A -- top 10 produktów po wartości sprzedaży

Dla produktów policz:

- ProductID,
- ProductName,
- łączną liczbę sprzedanych sztuk,

- łączną wartość sprzedaży.

Wykorzystaj kolekcje:

- orderdetails,
- products.

Przed zapytaniem może być potrzebny indeks:

```
CREATE INDEX idx_products_productid  
ON `north0`._default.products(ProductID);
```

```
CREATE INDEX idx_orderdetails_productid  
ON `north0`._default.orderdetails(ProductID);
```

Jeżeli indeks już istnieje, Couchbase zwróci komunikat o istniejącym indeksie --- to nie jest błąd.

Wariant B -- top 10 klientów po wartości zakupów

Dla klientów policz:

- CustomerID,
- CompanyName,
- liczbę zamówień,
- łączną wartość zakupów.

Wykorzystaj kolekcje:

- orders,
- orderdetails,
- customers.

W komentarzu napisz

- Który produkt albo klient ma najwyższą wartość sprzedaży?
- Czy wynik jest łatwy do biznesowej interpretacji?
- Czy zapytanie bardziej przypomina klasyczny SQL/BI, czy pracę z dokumentami JSON?

Odpowiedź

Wybrano wariant A

```
SELECT
  p.ProductID,
  p.ProductName,
  SUM(od.Quantity) AS total_quantity,
  ROUND(SUM(od.UnitPrice * od.Quantity * (1 - IFMISSINGORNULL(od.Discount, 0))),
2) AS total_sales
FROM `north0`._default.orderdetails AS od
JOIN `north0`._default.products AS p
  ON od.ProductID = p.ProductID
WHERE od.ProductID IS NOT MISSING
  AND p.ProductID IS NOT MISSING
GROUP BY p.ProductID, p.ProductName
ORDER BY total_sales DESC
LIMIT 10;
```

Wynik:

ProductID	ProductName	total_quantity	total_sales
38	Côte de Blaye	623	141396.73
29	Thüringer Rostbratwurst	746	80368.67
59	Raclette Courdavault	1496	71155.70
62	Tarte au sucre	1083	47234.97
60	Camembert Pierrot	1577	46825.48
56	Gnocchi di nonna Alice	1263	42593.06
51	Manjimup Dried Apples	886	41819.65
17	Alice Mutton	978	32698.38
18	Carnarvon Tigers	539	29171.87
28	Rössle Sauerkraut	640	25696.64

Komentarz:

Największą wartość sprzedaży ma produkt **Côte de Blaye** o **ProductID = 38**. Łączna wartość jego sprzedaży wyniosła **141396.73**, mimo że sprzedano **623** sztuki.

Wynik jest łatwy do interpretacji biznesowej, ponieważ od razu pokazuje, które produkty generują największą wartość sprzedaży. Widać też, że największa wartość sprzedaży nie musi oznaczać największej liczby sprzedanych sztuk, ponieważ wpływ ma także cena jednostkowa produktu.

Zapytanie bardziej przypomina klasyczne zapytanie SQL/BI, ponieważ używa **JOIN**, **GROUP BY**, **SUM** i **ORDER BY**. Elementem charakterystycznym dla Couchbase jest praca na kolekcjach dokumentów oraz użycie **IFMISSINGORNULL**, które zabezpiecza obliczenia na wypadek braku wartości rabatu.

6. EXPLAIN i refleksja końcowa -- 2 pkt

Cel

Nie wystarczy wiedzieć, że zapytanie działa. Trzeba jeszcze rozumieć, w jaki sposób baza je wykonuje.

Część A -- plan dla zapytania z JOIN

Wybierz zapytanie z zadania 3 albo 5 i uruchom je z EXPLAIN (albo za pomocą przycisku w menu).

Przykład:

```
EXPLAIN
SELECT ...
```

Odpowiedź

Do analizy planu wybrano zapytanie z **JOIN**, które liczy wartość zamówień na podstawie kolekcji **orders** i **orderdetails**.

```
EXPLAIN
SELECT
  o.OrderID,
  o.CustomerID,
  ROUND(SUM(od.UnitPrice * od.Quantity * (1 - IFMISSINGORNULL(od.Discount, 0))),
2) AS order_value,
  COUNT(1) AS items_count
FROM `north0`._default.orders AS o
JOIN `north0`._default.orderdetails AS od
  ON o.OrderID = od.OrderID
WHERE o.OrderID IS NOT MISSING
  AND od.OrderID IS NOT MISSING
GROUP BY o.OrderID, o.CustomerID
ORDER BY order_value DESC
LIMIT 10;
```

Wyniki

Najważniejsze elementy planu:

- **IndexScan3** na indeksie **idx_orders_orderid** dla kolekcji **orders**,
- **Fetch** dla pobrania dokumentów z kolekcji **orders**,
- **NestedLoopJoin**, czyli połączenie z kolekcją **orderdetails**,
- **IndexScan3** na indeksie **idx_orderdetails_orderid** dla kolekcji **orderdetails**,
- **Fetch** dla dokumentów z **orderdetails**,
- **InitialGroup**, **IntermediateGroup**, **FinalGroup**, czyli agregacja wartości zamówienia i liczby pozycji,
- **Order**, czyli sortowanie po **order_value**,
- **Limit**, czyli ograniczenie wyniku do 10 rekordów.

Komentarz

W planie zapytania z **JOIN** widać, że Couchbase korzysta z indeksów na polu **OrderID** w obu kolekcjach. Najpierw pobierane są dokumenty z **orders**, a następnie przez **NestedLoopJoin** dołączane są pasujące dokumenty z **orderdetails**. Na końcu baza grupuje dane po zamówieniu, liczy wartość zamówienia, sortuje wyniki malejąco i zwraca tylko 10 największych zamówień.

Część B -- plan dla zapytania z UNNEST

Wybierz zapytanie z zadania 4 i uruchom je z EXPLAIN.

Odpowiedź

Do analizy planu z **UNNEST** wybrano zapytanie działające na kolekcji **orders_nested**.

```
EXPLAIN
SELECT
  n.OrderID,
  n.CustomerID,
  ROUND(SUM(item.UnitPrice * item.Quantity * (1 - IFMISSINGORNULL(item.Discount,
0))), 2) AS order_value,
  COUNT(1) AS items_count
FROM `north0`._default.orders_nested AS n
UNNEST n.items AS item
WHERE n.OrderID IS NOT MISSING
GROUP BY n.OrderID, n.CustomerID
ORDER BY order_value DESC
LIMIT 10;
```

Wyniki

Najważniejsze elementy planu:

- **IndexScan3** na indeksie **idx_orders_nested_orderid** dla kolekcji **orders_nested**,
- **Fetch**, czyli pobranie pełnych dokumentów zamówień,
- **Filter**, który sprawdza m.in. czy **OrderID** istnieje oraz czy **items** jest tablicą,
- **Unnest**, czyli rozbiecie tablicy **items** na osobne rekordy,
- **InitialGroup**, **IntermediateGroup**, **FinalGroup**, czyli agregacja wartości zamówienia i liczby pozycji,
- **Order**, czyli sortowanie po **order_value**,
- **Limit**, czyli ograniczenie wyniku do 10 rekordów.

Komentarz:

W planie zapytania z **UNNEST** baza pracuje tylko na jednej kolekcji: **orders_nested**. Najpierw używa indeksu na **OrderID**, potem pobiera dokumenty, a następnie rozбивa tablicę **items** na osobne rekordy. W tym planie nie ma **NestedLoopJoin**, ponieważ pozycje zamówienia są już zapisane wewnątrz dokumentu.

Część C -- porównanie

Porównaj oba plany na poziomie ogólnym.

Nie opisuj całego planu. Wystarczy wskazać, z jakich indeksów korzysta zapytanie, czy pojawia się JOIN albo UNNEST, oraz gdzie widać agregację i sortowanie.

Wskaż elementy typu:

- IndexScan,
- Fetch,
- NestedLoopJoin,
- Unnest,
- Group,
- Order,
- Limit.

W komentarzu końcowym napisz

Odpowiedz w kilku zdaniach:

- Co było największą różnicą między Couchbase a klasyczną bazą relacyjną?
- Dlaczego indeksy są tak ważne w Couchbase?
- Co pokazało porównanie JOIN i UNNEST?
- Czy dokumentowy model danych wyklucza analizę i raportowanie?
- Gdybyś projektował system zamówień, kiedy rozważyłbyś zagnieżdżenie pozycji zamówienia w dokumencie zamówienia?

Odpowiedź

Plan zapytania z **JOIN** i plan zapytania z **UNNEST** mają podobną końcówkę: w obu przypadkach pojawia się agregacja, sortowanie i ograniczenie wyniku przez **LIMIT 10**.

Największa różnica jest w sposobie dojścia do pozycji zamówienia. W zapytaniu z **JOIN** baza musi połączyć dwie kolekcje: **orders** i **orderdetails**. Dlatego w planie pojawia się **NestedLoopJoin** oraz osobny odczyt z drugiej kolekcji. W zapytaniu z **UNNEST** baza nie łączy kolekcji, tylko rozbija tablicę **items** znajdującą się w dokumencie **orders_nested**.

Oznacza to, że **JOIN** jest potrzebny przy modelu niezagnieżdżonym, gdzie dane są w osobnych kolekcjach. **UNNEST** jest używany przy modelu zagnieżdżonym, gdzie pozycje zamówienia są zapisane w tablicy wewnątrz dokumentu.

Komentarz końcowy

Największą różnicą między Couchbase a klasyczną bazą relacyjną jest sposób przechowywania danych. W Couchbase dane są dokumentami JSON, które mogą mieć zagnieżdżone obiekty i tablice. W relacyjnej bazie dane są zwykle rozdzielone na tabele.

Indeksy są w Couchbase bardzo ważne, ponieważ bez odpowiedniego indeksu część zapytań może się nie wykonać albo działać nieefektywnie. Dlatego indeksy trzeba projektować pod konkretne zapytania.

Porównanie **JOIN** i **UNNEST** pokazało, że oba podejścia mogą dać ten sam wynik biznesowy, ale działają inaczej. **JOIN** łączy dane z osobnych kolekcji, a **UNNEST** rozbija tablicę znajdującą się wewnątrz jednego dokumentu.

Dokumentowy model danych nie wyklucza analiz i raportowania. Nadal można wykonywać agregacje, sortowanie, grupowanie i liczyć wartości biznesowe. Trzeba jednak dobrze znać strukturę dokumentów i wiedzieć, kiedy używać **JOIN**, a kiedy **UNNEST**.

Zagnieżdżenie pozycji zamówienia w dokumencie rozważyłabym wtedy, gdy aplikacja często pobiera całe zamówienie razem z jego pozycjami. Jeśli natomiast pozycje zamówień mają być często analizowane osobno, np. według produktów, wygodniejsze mogą być osobne kolekcje.

Zadanie dodatkowe dla chętnych

Materializacja KPI klienta

Utwórz kolekcję `customer_kpis`, a następnie zapisz do niej gotowe dokumenty z metrykami klienta:

- `CustomerID`,
- `CompanyName`,
- `Revenue`,
- `OrdersCount`.

Następnie wykonaj zapytanie raportowe na kolekcji `customer_kpis`.

Uwaga: wskazówki do rozwiązania zadania znajdziesz w ściądze.

W komentarzu napisz:

- czym różni się liczenie raportu „w locie” od czytania gotowej kolekcji KPI,
- kiedy takie podejście może być użyteczne,
- jakie jest ryzyko materializowania wyników, jeśli dane źródłowe się zmieniają.

referencje